

4. Заполняем БД тестовыми данными.

Изучаем БД «Авиаперевозки»

- Как сгенерировать тестовые данные?
 - Как разобраться в структуре БД
 - Отдельные интересные решения
 - Запросы с JOIN, алгоритмы JOIN
 - Довольно-таки сложные запросы

```
SELECT random()::text;
```

```
SELECT encode(sha256(random()::text::bytea), 'hex');
```

```
CREATE OR REPLACE FUNCTION rnd_cyrillic_fix(length INT) RETURNS TEXT AS $$
DECLARE
    result TEXT := "";
    chars_upper TEXT := 'АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ';
    chars_lower TEXT := 'абвгдеёжзийклмнопрстуфхцчшщъыьэюя';
    chars_length_upper INT := length(chars_upper);
    chars_length_lower INT := length(chars_lower);
    i INT;
BEGIN
    result := substr(chars_upper, (floor(random() * chars_length_upper) + 1)::INT, 1);
    FOR i IN 2..length LOOP
        result := result || substr(chars_lower, (floor(random() * chars_length_lower) + 1)::INT,
1);
    END LOOP;
    RETURN result;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE FUNCTION randint(from_ integer, to_ integer)
    RETURNS integer
AS $$
BEGIN
    IF from_ > to_ THEN
        RAISE EXCEPTION 'from_ > to_';
    END IF;
    RETURN floor(random() * (to_ - from_)) + from_;
END;
$$ LANGUAGE PLPGSQL;
```

```
CREATE OR REPLACE FUNCTION randstr(len integer)
  RETURNS text
AS $$
DECLARE
  result text := '';
  cnt integer := 0;
BEGIN
  IF len < 0 THEN
    RAISE EXCEPTION 'len < 0';
  END IF;
  WHILE cnt < len LOOP
    BEGIN
      result := result || chr(randint(1, 0x10000));
      cnt := cnt + 1;
    EXCEPTION
      WHEN others THEN NULL;
    END;
  END LOOP;
  RETURN result;
END;
$$ LANGUAGE PLPGSQL;

SELECT randstr(15) FROM generate_series(1,20);
```

```
CREATE OR REPLACE FUNCTION randstr(len integer, alphabet text[])
    RETURNS text
AS $$
DECLARE
    result text := '';
BEGIN
    IF len < 0 THEN
        RAISE EXCEPTION 'len < 0';
    END IF;
    FOR i IN 1..len LOOP
        result := result || alphabet[randint(1, cardinality(alphabet) + 1)];
    END LOOP;
    RETURN result;
END;
$$ LANGUAGE PLPGSQL;
```

```
SELECT randstr(10, '{а,б,в,г,д,ё,ы,ь,э,ю,я}') FROM generate_series(1,20);
```

```
CREATE OR REPLACE FUNCTION generate_random_string(len integer)
RETURNS TEXT AS $$
DECLARE
    chars TEXT :=
'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789';
BEGIN
    IF len < 1 THEN
        RETURN "";
    END IF;

    RETURN (
        SELECT string_agg(
            substr(chars, (floor(random() * length(chars)) + 1)::integer, 1),
            ""
        )
        FROM generate_series(1, len)
    );
END;
$$ LANGUAGE plpgsql;
```

Случайная буква

```
SELECT  
  generate_series (1,1000),  
  substring(  
    'абвгде',  
    1+floor (6*random())::int ,  
    1  
  );
```

Последовательные числа (id), случайные буквы (name)

Как получить случайную строку?

Попробуем:

```
SELECT generate_series(1,1000),  
       (SELECT string_agg(x, '') FROM (  
         SELECT substring(  
           'абвгде',  
           1+floor (6*random())::int ,  
           1  
         ) x FROM generate_series(1,3)  
       ) g  
);
```

Что получилось? Почему?

Другой вариант

Попробуем:

```
SELECT id, string_agg(x, '') FROM (  
  SELECT substring(  
    'абвгде',  
    1+floor (6*random())::int ,  
    1  
  ) x, 1+floor(1000*random())::int id  
  FROM generate_series(1,6000)  
) g GROUP BY id ORDER BY id;
```

А как теперь? Какие плюсы и минусы этого варианта?

Теперь заполним таблицы.

```
INSERT INTO student  
SELECT generate_series(1,1000000),  
       random_string(10);
```

Простой поиск

Посмотрим план запроса

```
EXPLAIN ANALYZE  
SELECT *  
FROM student  
WHERE name = 'бгдггбаегг';
```

А если

```
CREATE INDEX student_name ON student (name);
```

???

Что такое «план запроса»

Посмотрим план запроса

```
Gather  (cost=1000.00..13561.43 rows=1 width=25) (actual
time=44.957..50.304 rows=0 loops=1)
  Workers Planned: 2
  Workers Launched: 2
    -> Parallel Seq Scan on student  (cost=0.00..12561.33
rows=1 width=25) (actual time=40.057..40.058 rows=0
loops=3)
      Filter: (name = 'Бгдггбаера'::text)
      Rows Removed by Filter: 333333

Planning Time: 0.209 ms
Execution Time: 50.330 ms
```

Что дал индекс

Посмотрим план запроса

```
Index Scan using student_name on student  
(cost=0.42..8.44 rows=1 width=25) (actual  
time=0.035..0.035 rows=0 loops=1)
```

```
Index Cond: (name = 'бгдггбаера'::text)
```

```
Planning Time: 0.226 ms
```

```
Execution Time: 0.053 ms
```

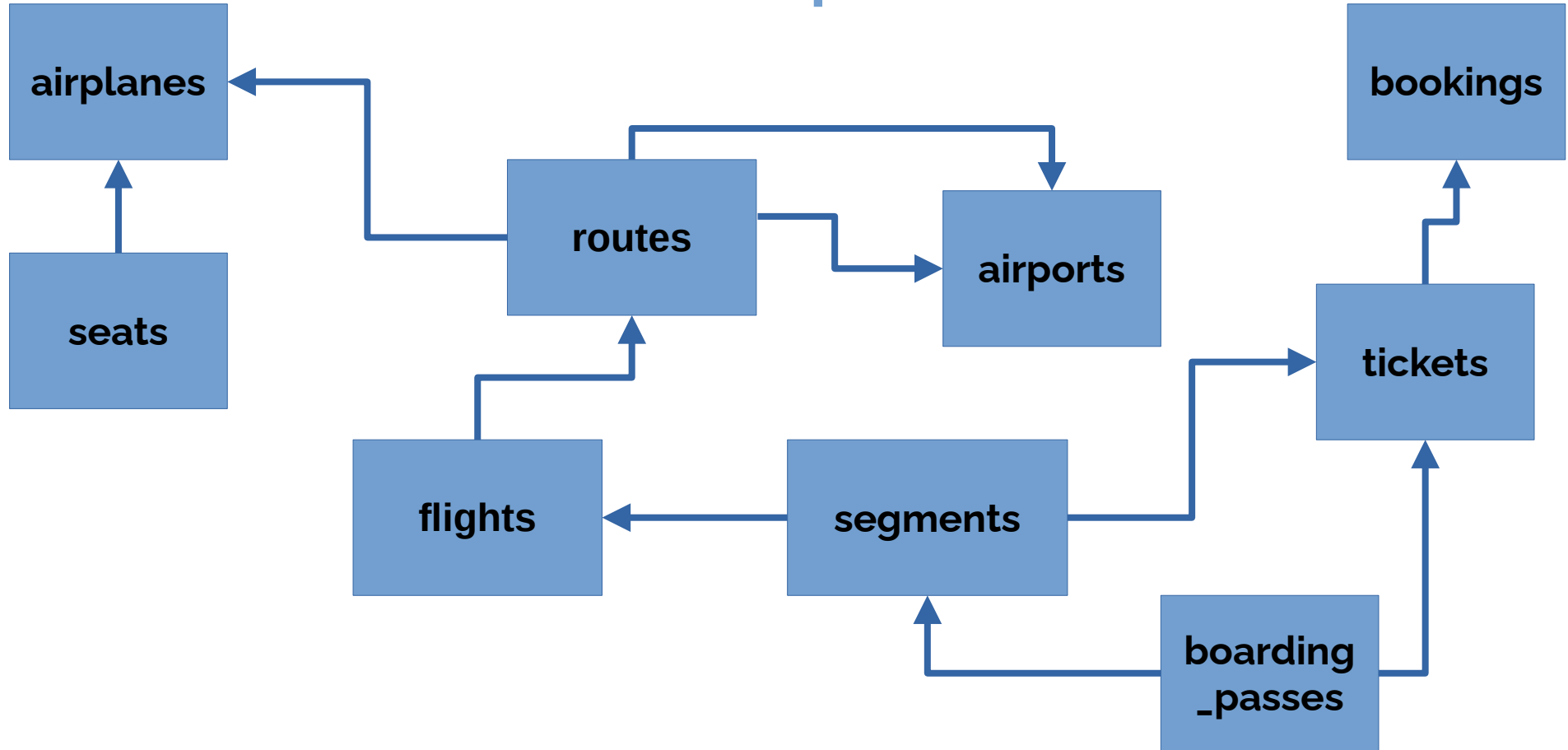
«Представления»

```
CREATE VIEW имя AS SELECT ...;  
SELECT * FROM имя;
```

View == Представление

VIEW	MATERIALIZED VIEW
Сохраненный запрос, который можно использовать как таблицу	Сохраненный результат запроса, который можно использовать как таблицу
• Упрощение кода (не повторять) • Управление доступом	• Хранение промежуточных результатов для повышения производительности

«Авиаперевозки»



Изучаем таблицы

```
demo=# \d
```

Список отношений			
Схема	Имя	Тип	Владелец
bookings	airplanes	представление	postgres
bookings	airplanes_data	таблица	postgres
bookings	airports	представление	postgres
bookings	airports_data	таблица	postgres
bookings	boarding_passes	таблица	postgres
bookings	bookings	таблица	postgres
bookings	flights	таблица	postgres
bookings	flights_flight_id_seq	последовательность	postgres
bookings	routes	таблица	postgres
bookings	seats	таблица	postgres
bookings	segments	таблица	postgres
bookings	tickets	таблица	postgres
bookings	timetable	представление	

(13 строк)

```
demo=# show search_path;  
search_path
```

```
bookings, "$user", public  
(1 строка)
```

Изучаем таблицы

```
demo=# \d+ airplanes
```

Представление "bookings.airplanes"

Столбец	Тип	Правило сортировки	Допустимость NULL	По умолчанию	Хранилище	Описание
airplane_code	character(3)				extended	Airplane code, IATA
model	text				extended	Airplane model
range	integer				plain	Maximum flight range, km
speed	integer				plain	Cruise speed, km/h

Определение представления:

```
SELECT airplane_code,  
       model ->> lang() AS model,  
       range,  
       speed  
FROM airplanes_data ml;
```

```
demo=# \d+ airplanes_data
```

Таблица "bookings.airplanes_data"

Столбец	Тип	Правило сортировки	Допустимость NULL	По умолчанию	Хранилище	Сжатие	Цель для статистики	Описание
airplane_code	character(3)		not null		extended			Airplane code, IATA
model	jsonb		not null		extended			Airplane model
range	integer		not null		plain			Maximum flight range, km
speed	integer		not null		plain			Cruise speed, km/h

Индексы:

```
"airplanes_data_pkey" PRIMARY KEY, btree (airplane_code)
```

Ограничения-проверки:

```
"airplanes_data_range_check" CHECK (range > 0)
```

```
"airplanes_data_speed_check" CHECK (speed > 0)
```

Ссылки извне:

```
TABLE "routes" CONSTRAINT "routes_airplane_code_fkey" FOREIGN KEY (airplane_code) REFERENCES airplanes_data(airplane_code)
```

```
TABLE "seats" CONSTRAINT "seats_airplane_code_fkey" FOREIGN KEY (airplane_code) REFERENCES airplanes_data(airplane_code) ON DELETE CASCADE
```

Метод доступа: heap

Функция!

Функция lang()

Как устроена внутри?

```
SELECT prosrc  
  FROM pg_proc  
 WHERE proname = 'lang';
```

\df+ lang

\ef lang

Что там внутри?

```
CREATE OR REPLACE FUNCTION bookings.lang()
```

```
  RETURNS text
```

```
  LANGUAGE sql
```

```
  STABLE
```

```
  RETURN current_setting('bookings.lang'::text, true)
```

```
SHOW bookings.lang;
```

```
SET bookings.lang = 'en';
```

JSON

```
demo=# SELECT * FROM airplanes_data LIMIT 1;
```

airplane_code	model	range	speed
32N	{"en": "Aerobus A320neo", "ru": "Аэробус A320neo"}	6500	830

(1 строка)

model ->> lang()

Необычный способ интернационализации.
Но имеющий право на существование

AIRPORTS

```
\d airports
```

```
...  
coordinates point  
...
```

point — геометрический тип данных: (x,y)

```
SELECT coordinates FROM airports;
```

FLIGHTS

\d+ routes

CHECK CONSTRAINTS

\d+ flights

SQL-практика

Задачи:

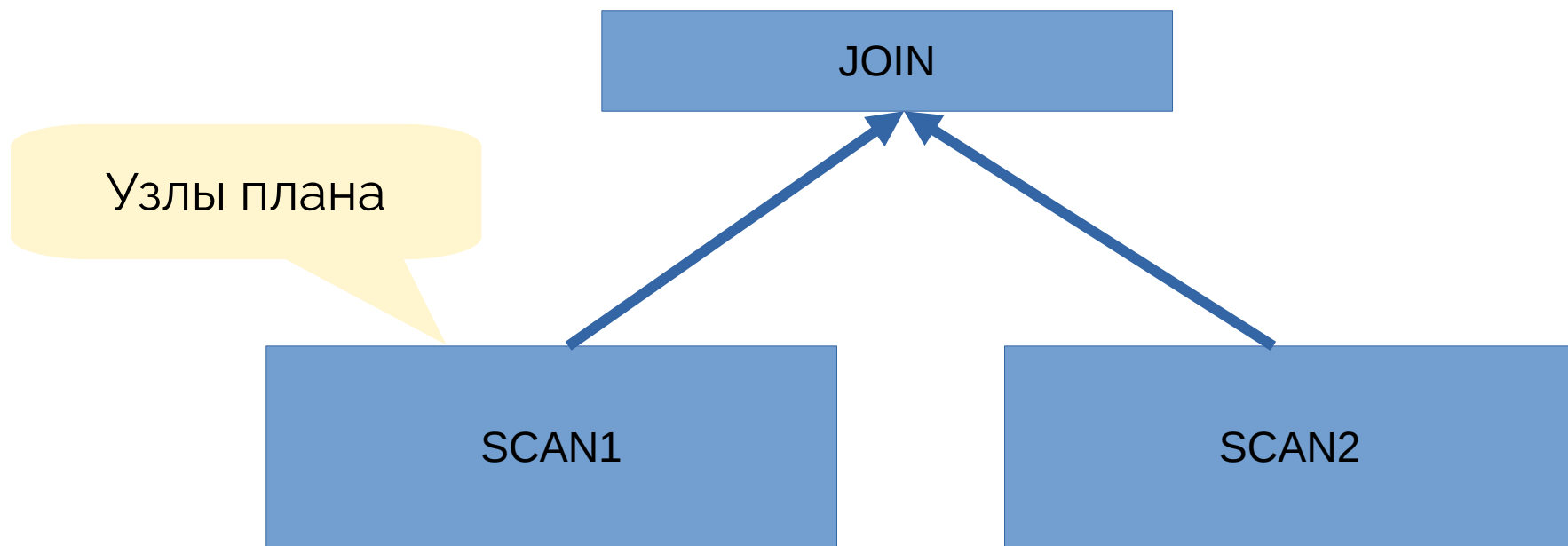
- Получим список всех рейсов.
- Каков план этого запроса?
- Какие были рейсы в Красноярск? (KJA)
- Посмотрим план этого запроса (EXPLAIN ANALYZE)
- Как влияет наличие индексов на план запроса?

JOIN, подзапросы

Задачи:

- Получим список пассажиров, летевших на рейсе №1185 (flight_id)
 - Какой план? Какие индексы используются?
 - Какой алгоритм JOIN ?
- Получим список пассажиров, летевших на самом последнем непустом рейсе
 - Какой план?

Как устроен план?



Блокировки

```
BEGIN;
```

```
UPDATE flights  
SET status = 'Cancelled'  
WHERE flight_id = 1185;
```

```
COMMIT;
```

```
UPDATE flights  
SET status =  
'Cancelled'  
WHERE flight_id = 1185;  
ждём...
```

проскочило!

Изоляция транзакций

Read Committed

```
BEGIN;
```

```
SELECT status  
FROM flights WHERE id=1185;  
..Cancelled..
```

```
UPDATE flights  
SET status = 'Scheduled'  
WHERE flight_id = 1185;
```

```
SELECT status  
FROM flights WHERE id=1185;  
..Scheduled..
```

```
COMMIT;
```

Изоляция транзакций

Repeatable Read

```
BEGIN ISOLATION LEVEL  
    REPEATABLE READ;
```

```
SELECT status  
FROM flights WHERE id=1185;  
..Scheduled..
```

```
UPDATE flights  
SET status = 'Cancelled'  
WHERE flight_id = 1185;
```

```
SELECT status  
FROM flights WHERE id=1185;  
..Scheduled..
```

```
COMMIT;
```

Ho he COBCEM...

```
BEGIN ISOLATION LEVEL  
    REPEATABLE READ;
```

```
SELECT count (*)  
FROM flights  
WHERE status = 'Scheduled';
```

414

```
SELECT count (*)  
FROM flights  
WHERE status = 'Scheduled';
```

415

```
COMMIT;
```

```
UPDATE flights  
SET status = 'Scheduled'  
WHERE flight_id = 1185;
```

SERIALIZABLE

```
BEGIN ISOLATION LEVEL  
    SERIALIZABLE;
```

```
SELECT count (*)  
FROM flights  
WHERE status = 'Cancelled';
```

415

```
SELECT count (*)  
FROM flights  
WHERE status = 'Cancelled';
```

415

```
COMMIT;
```

```
UPDATE flights  
SET status = 'Cancelled'  
WHERE flight_id = 1185;
```

```
SELECT count (*)  
FROM flights  
WHERE status = 'Cancelled';
```

414

Другой подход....

```
BEGIN;
```

```
SELECT *  
FROM flights FOR UPDATE;
```

```
SELECT count (*)  
FROM flights  
WHERE status = 'Cancelled';
```

415

```
COMMIT;
```

```
UPDATE flights  
SET status = 'Cancelled'  
WHERE flight_id = 1185;  
ждём... .
```

Проскочило!

КОНЕЦ

К следующему разу:

Знать «авиаперевозки» как свои 5 пальцев.

Что будем делать:

Продолжим изучать MVCC, блокировки, вакуум